



Conception et mise en place d'un système de détection et de réponse aux attaques ransomware dans un environnement contrôlé

Codjo Fréjus Loïc SANGRONIO

Licence Informatique — Option Sécurité Informatique

24 juin 2026

Encadreur :

Ing. Guy KPEDJO

Maître de stage :

[Nom du maître de stage]



RansomShield

2026-07-03

[OUVERTURE — 0 :00]

Monsieur le Président du jury, Mesdames et Messieurs les membres du jury, bonjour. Je vous remercie pour l'occasion qui m'est donnée de présenter mon travail portant sur la conception et la mise en place d'un système de détection et de réponse aux attaques ransomware dans un environnement contrôlé : RansomShield. Cette présentation durera dix minutes, suivie d'une démonstration live de cinq minutes sur des machines virtuelles réelles. Je reste à votre disposition pour vos questions à la fin.



- 1. Introduction Générale
- 2. Revue de littérature
- 3. Analyse et Conception
- 4. Réalisation et Résultats
- 5. Conclusion et Perspectives

Présentation : 10 min

Diapositives : 21

Démo live : 5 min

VMs test : 3 KVM

2026-07-03

Sommaire

[SOMMAIRE — 0 :30]

Ma présentation s'articule en cinq parties : le contexte et la problématique, la revue de littérature, la conception de la solution, sa réalisation avec les résultats obtenus, puis la conclusion. Je terminerai par une courte démonstration du système en fonctionnement.

Sommaire

- 1. Introduction Générale
- 2. Revue de littérature
- 3. Analyse et Conception
- 4. Réalisation et Résultats
- 5. Conclusion et Perspectives

Présentation : 10 min

Diapositives : 21

Démo live : 5 min

VMs test : 3 KVM



01

🎯 Introduction Générale



2026-07-03

RansomShield
└ Introduction Générale

01 🎯

Contexte

L'évolution rapide du numérique expose les organisations aux ransomwares : des logiciels malveillants qui rendent les données inaccessibles puis exigent une rançon.

- +73 % d'attaques signalées en 2023 (ENISA)
- Délai moyen de détection **sans outil dédié : 21 jours**
- Coût moyen par attaque : > **1 million de dollars**
- Cibles : PME, hôpitaux, administrations, universités

Problématique

Les terminaux constituent des points d'entrée sensibles, tandis que les antivirus basés sur les signatures restent insuffisants face aux variantes nouvelles ou inconnues. La nécessité de détecter automatiquement un comportement ransomware sur un parc de machines hétérogène — et de déclencher une réponse maîtrisée **sans déployer de vrai malware** — constitue le défi central de ce travail.

2026-07-03

RansomShield
└─ Introduction Générale

└─ Introduction Générale

[CONTEXTE & PROBLÉMATIQUE — 1 :00]

Les organisations dépendent aujourd'hui fortement de leurs systèmes informatiques, ce qui les expose de plus en plus aux cyberattaques — en particulier au ransomware, ce logiciel malveillant qui chiffre les données puis exige une rançon. Selon l'ENISA, ces attaques ont augmenté de 73 % en 2023 ; sans outil dédié, il faut en moyenne 21 jours pour les détecter, pour un coût qui dépasse souvent le million de dollars. Le problème, c'est que les antivirus classiques, basés sur des signatures, ne suffisent plus face à des comportements inconnus. D'où la question centrale de ce travail : comment détecter et répondre à un comportement ransomware, dans un environnement contrôlé, sans déployer de vrai malware ?

Introduction Générale	
Contexte	
L'évolution rapide du numérique expose les organisations aux ransomwares : des logiciels malveillants qui rendent les données inaccessibles puis exigent une rançon.	
▪ +73 % d'attaques signalées en 2023 (ENISA)	
▪ Délai moyen de détection sans outil dédié : 21 jours	
▪ Coût moyen par attaque : > 1 million de dollars	
▪ Cibles : PME, hôpitaux, administrations, universités	
Problématique	
Les terminaux constituent des points d'entrée sensibles, tandis que les antivirus basés sur les signatures restent insuffisants face aux variantes nouvelles ou inconnues. La nécessité de détecter automatiquement un comportement ransomware sur un parc de machines hétérogènes — et de déclencher une réponse maîtrisée sans déployer de vrai malware — constitue le défi central de ce travail.	

Objectif général

Concevoir et mettre en œuvre un **système de détection et de réponse aux attaques ransomware** basé sur la surveillance comportementale des fichiers et des processus, avec une console de supervision traçable.

Objectifs spécifiques

- Surveiller les événements fichiers en temps réel sur les terminaux
- Analyser les comportements et calculer un score de risque configurable
- Générer automatiquement des alertes et des incidents de sécurité
- Proposer et exécuter des actions de protection avec **contrôle humain**
- Sécuriser la console par une authentification forte (2FA TOTP RFC 6238)
- Valider le pipeline à l'aide de scénarios d'attaque représentatifs

2026-07-03

RansomShield
└─ Introduction Générale

└─ Introduction Générale

[OBJECTIFS — 2 :30]

L'objectif général est de concevoir un système de détection et de réponse aux attaques ransomware, basé sur la surveillance comportementale des fichiers et des processus. Concrètement, il s'agit de surveiller les événements en temps réel, de calculer un score de risque, de générer automatiquement des alertes et des incidents, de proposer des actions de protection avec un contrôle humain systématique, de sécuriser la console par une double authentification, et de valider le tout sur des scénarios d'attaque réalistes.

Introduction Générale	
2026-07-03	Objectif général
	Concevoir et mettre en œuvre un système de détection et de réponse aux attaques ransomware basé sur la surveillance comportementale des fichiers et des processus, avec une console de supervision traçable.
	Objectifs spécifiques
	▪ Surveiller les événements fichiers en temps réel sur les terminaux ▪ Analyser les comportements et calculer un score de risque configurable ▪ Générer automatiquement des alertes et des incidents de sécurité ▪ Proposer et exécuter des actions de protection avec contrôle humain ▪ Sécuriser la console par une authentification forte (2FA TOTP RFC 6238) ▪ Valider le pipeline à l'aide de scénarios d'attaque représentatifs

Organisation du mémoire

Chapitre I Revue de littérature

Sécurité informatique, logiciels malveillants, ransomwares, antivirus, IDS, EDR : positionnement de RansomShield.

Chapitre II Analyse et Conception

Besoins fonctionnels et non fonctionnels, architecture générale, modèle de données, processus de détection et mesures de sécurité.

Chapitre III Réalisation et Résultats

Implémentation des composants, scénarios de validation, résultats obtenus, limites et perspectives.

Mémoire rédigé en $\text{\LaTeX}$ selon le canevas IFRI — Licence Informatique, option Sécurité Informatique — Année 2023–2024

2026-07-03

RansomShield
└─ Introduction Générale

└─ Organisation du mémoire

[ORGANISATION DU MÉMOIRE — 3 :15]

Le mémoire suit le canevas IFRI en trois chapitres : la revue de littérature, l'analyse et la conception du système, puis sa réalisation et ses résultats. Voyons d'abord les fondements théoriques.

Organisation du mémoire

Chapitre I	Chapitre II	Chapitre III
Revue de littérature	Analyse et Conception	Réalisation et Résultats
Sécurité informatique, logiciels malveillants, ransomwares, antivirus, IDS, EDR : positionnement de RansomShield.	Besoins fonctionnels et non fonctionnels, architecture générale, modèle de données, processus de détection et mesures de sécurité.	Implémentation des composants, scénarios de validation, résultats obtenus, limites et perspectives.

Mémoire rédigé en $\text{\LaTeX}$ selon le canevas IFRI — Licence Informatique, option Sécurité Informatique — Année 2023–2024



02



Revue de littérature



2026-07-03

RansomShield
— Revue de littérature

02



 Revue de littérature

Définition d'un ransomware

Un ransomware est un logiciel malveillant qui vise à bloquer l'accès à un système ou à des données, souvent par chiffrement, puis à réclamer une rançon à la victime. (NIST SP 1800-25, CISA)

Comportements caractéristiques — détectables avant les dommages irréversibles

- Renommage massif de fichiers avec extension suspecte (.locked, .enc, .crypt...)
- Création d'une note de rançon (HOW_TO_DECRYPT.txt)
- Suppression des clichés instantanés VSS (vssadmin delete shadows /all)
- Utilisation d'outils légitimes détournés — LOLBins (bcdedit, wmic, vssadmin)
- Suppression massive de fichiers, accès réseau inhabituel

Ces signaux constituent la base de la **détection comportementale**.

2026-07-03

RansomShield
└─ Revue de littérature

└─ Revue de littérature

[DÉFINITION & COMPORTEMENTS — 3 :50]

Selon le NIST, un ransomware est un logiciel malveillant qui bloque l'accès aux données, le plus souvent par chiffrement, pour ensuite réclamer une rançon. Ce qui compte pour nous, c'est que ce type d'attaque laisse des signes avant que les dégâts ne soient irréversibles : renommage massif de fichiers, apparition d'extensions suspectes comme .locked, création d'une note de rançon, suppression des sauvegardes système, ou encore utilisation détournée d'outils légitimes. Ce sont ces signaux qui constituent la base de notre détection comportementale.

Revue de littérature
Définition d'un ransomware
Un ransomware est un logiciel malveillant qui vise à bloquer l'accès à un système ou à des données, souvent par chiffrement, puis à réclamer une rançon à la victime. (NIST SP 1800-25, CISA)
Comportements caractéristiques — détectables avant les dommages irréversibles
▪ Renommage massif de fichiers avec extension suspecte (.locked, .enc, .crypt...) ▪ Création d'une note de rançon (HOW_TO_DECRYPT.txt) ▪ Suppression des clichés instantanés VSS (vssadmin delete shadows /all) ▪ Utilisation d'outils légitimes détournés — LOLBins (bcdedit, wmic, vssadmin) ▪ Suppression massive de fichiers, accès réseau inhabituel Ces signaux constituent la base de la détection comportementale .

Quelques solutions existantes

- **Antivirus** — détection par signatures statiques (ClamAV, Windows Defender)
- **SIEM génériques** — collecte de journaux sans corrélation comportementale
- **EDR commerciaux** — CrowdStrike Falcon, SentinelOne, Microsoft Defender XDR

Limites des solutions existantes

- ❌ Contournables par variantes nouvelles, obfuscation ou LOLBins
- ❌ Coût élevé des EDR commerciaux (hors portée académique)
- ❌ Absence d’outil open-source académique démontrable en conditions réelles

Notre réponse : RansomShield

Approche comportementale open-source, multi-plateforme, démontrable sur machines virtuelles réelles — **contrôle humain à chaque étape.**

2026-07-03

RansomShield
└─ Revue de littérature

└─ Revue de littérature

[SOLUTIONS EXISTANTES — 5 :15]

Les solutions existantes ont chacune leurs limites. Les antivirus classiques détectent par signatures, donc ils passent à côté des nouvelles variantes. Les SIEM collectent des journaux mais sans corrélation comportementale spécifique. Les EDR commerciaux, comme CrowdStrike ou SentinelOne, sont efficaces mais hors de portée pour un contexte académique. C’est dans cet espace que se positionne RansomShield : une approche comportementale, open-source, avec un contrôle humain garanti à chaque étape. Voyons maintenant comment ce système a été conçu.

Revue de littérature
Quelques solutions existantes
▪ Antivirus — détection par signatures statiques (ClamAV, Windows Defender) ▪ SIEM génériques — collecte de journaux sans corrélation comportementale ▪ EDR commerciaux — CrowdStrike Falcon, SentinelOne, Microsoft Defender XDR
Limites des solutions existantes
▪ ❌ Contournables par variantes nouvelles, obfuscation ou LOLBins ▪ ❌ Coût élevé des EDR commerciaux (hors portée académique) ▪ ❌ Absence d’outil open-source académique démontrable en conditions réelles
Notre réponse : RansomShield
Approche comportementale open-source, multi-plateforme, démontrable sur machines virtuelles réelles — contrôle humain à chaque étape.



Analyse et Conception

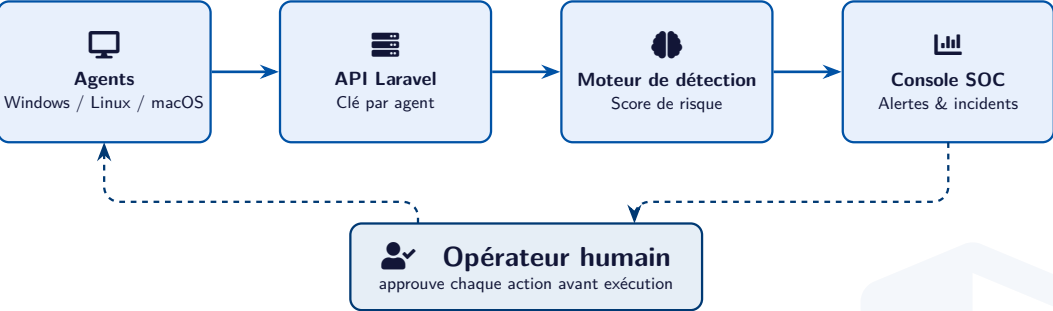


2026-07-03

RansomShield
— Analyse et Conception



Architecture générale de RansomShield

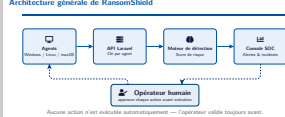


Aucune action n'est exécutée automatiquement — l'opérateur valide toujours avant.

2026-07-03

RansomShield
Analyse et Conception

Architecture générale de RansomShield



[ARCHITECTURE — 6 :15]

L'architecture repose sur un flux simple. Les agents Python, installés sur les machines surveillées, envoient les événements à l'API Laravel via une clé propre à chaque agent. Le moteur de détection analyse ces événements, calcule un score de risque, et déclenche une alerte puis un incident si le seuil est dépassé. L'opérateur consulte la console SOC et valide chaque action avant son exécution : c'est notre règle d'or, aucune action automatique sans validation humaine.

Les quatre composants du système

Agent Python

Surveille les dossiers via **Watchdog** / **inotify**. File locale SQLite (résilience réseau). Enrôlement par token à usage unique. Service systemd — démarrage automatique.

Découverte réseau

fping (ICMP) + ARP sweep sur les /24. Scan automatique toutes les 5 minutes. Aucun enrôlement sans **validation manuelle admin**.

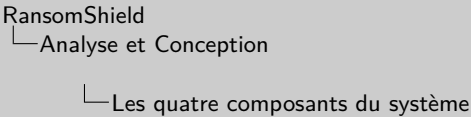
Console SOC Laravel

Dashboard temps réel (Chart.js, AJAX 30s). Alertes, incidents, timeline, file d'approbation. Notifications : web, e-mail, son, webhook.

Module de simulation

5 scénarios d'attaque, jusqu'à **22 événements**. Tous marqués is_simulation=true — nettoyage facile après chaque démo.

2026-07-03

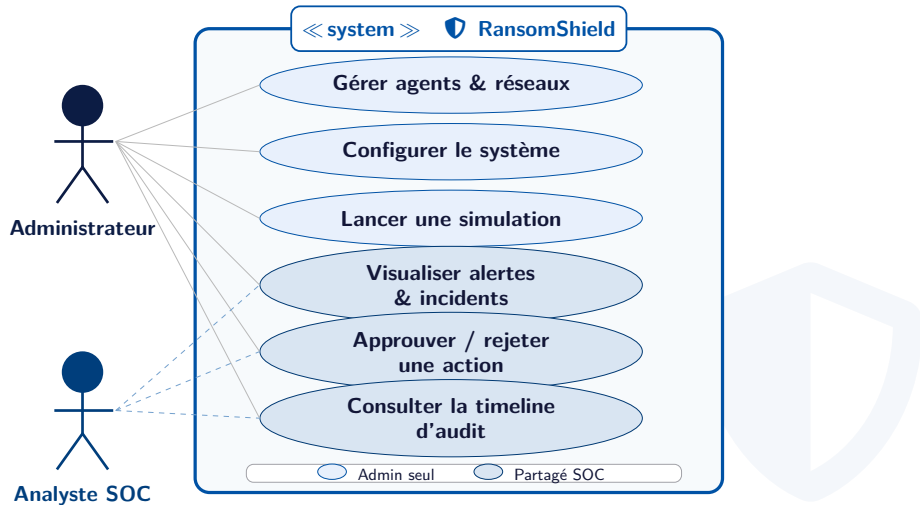


[LES 4 COMPOSANTS — 7 :15]

Le système repose sur quatre composants. L'agent Python surveille les fichiers en temps réel et conserve une file locale pour ne rien perdre en cas de coupure réseau. La console SOC centralise le suivi : alertes, incidents, et validation des actions. La découverte réseau scanne automatiquement les sous-réseaux, mais aucun agent ne peut s'enrôler sans validation manuelle de l'administrateur. Enfin, le module de simulation permet de rejouer des scénarios d'attaque réalistes, jusqu'à vingt-deux événements, sans jamais toucher à un vrai malware.

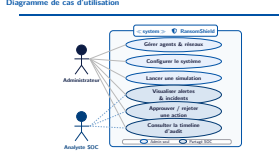
Les quatre composants du système			
	Agent Python		Console SOC Laravel
Surveille les dossiers via Watchdog / inotify. File locale SQLite (résilience réseau). Enrôlement par token à usage unique. Service systemd — démarrage automatique.		Dashboard temps réel (Chart.js, AJAX 30s). Alertes, incidents, timeline, file d'approbation. Notifications : web, e-mail, son, webhook.	
	Découverte réseau		Module de simulation
fping (ICMP) + ARP sweep sur les /24. Scan automatique toutes les 5 minutes. Aucun enrôlement sans validation manuelle admin.		5 scénarios d'attaque, jusqu'à 22 événements. Tous marqués is_simulation=true — nettoyage facile après chaque démo.	

Diagramme de cas d'utilisation



2026-07-03

RansomShield
— Analyse et Conception
— Diagramme de cas d'utilisation



[DIAGRAMME UML — 8 :45]

Le système distingue deux acteurs. L'administrateur gère les agents, configure le système et lance les simulations. L'analyste SOC, lui, se concentre sur le traitement opérationnel : il visualise les alertes et les incidents, approuve ou rejette les actions de protection, et consulte la timeline d'audit — mais n'a pas accès à la configuration.

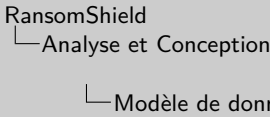
Modèle de données

Tables principales	
agents	enrôlement, état, risque
events	événements fichiers
alerts	score, niveau, signaux
incidents	fiche, statut, timeline
detection_rules	règles configurables
protection_actions	décisions SOC
audit_logs	traçabilité horodatée
simulation_runs	profils d'attaque

Relations clés
agents → events (1-N)
events → alerts (N-1)
alerts → incidents (N-1)
incidents → protection_actions (1-N)
incidents → audit_logs (1-N)
simulation_runs → events (1-N)

8 tables MySQL — pipeline complet : événement → alerte → incident → action → audit

2026-07-03



[MODÈLE DE DONNÉES — 9 :45]

Le modèle de données repose sur huit tables MySQL. Le pipeline se lit de gauche à droite : un agent génère des événements, qui déclenchent des alertes, qui créent des incidents, qui donnent lieu à des actions de protection — le tout tracé dans un journal d'audit. Chaque action est horodatée avec l'identité de l'opérateur qui l'a validée. Passons maintenant à la réalisation concrète.



Réalisation et Résultats



Backend & Base de données

- Laravel 11 / PHP 8.3
- MySQL 8.0 (Eloquent ORM)
- API REST sécurisée (clé per-agent)

Agent de surveillance

- Python 3.10
- Watchdog (inotify Linux)
- SQLite (file locale offline)
- psutil (processus)

Frontend

- Blade (templates Laravel)
- Chart.js (intégré local)
- AJAX toutes les 30 s

Infrastructure de test

- 3 VMs KVM/libvirt
- Réseau 10.20.0.0/24
- 2FA TOTP RFC 6238
- Token usage unique (enrôlement)

2026-07-03

RansomShield
Réalisation et Résultats
Choix techniques

[CHOIX TECHNIQUES — 11 :00]

Côté technique, le backend repose sur Laravel 11 et PHP 8.3, avec une base MySQL et une API sécurisée par une clé propre à chaque agent. L'agent de surveillance est écrit en Python avec la bibliothèque Watchdog, et conserve ses événements dans une file SQLite locale. Le frontend utilise Chart.js en local, sans dépendance externe. Les tests ont été menés sur trois machines virtuelles KVM, avec une authentification à double facteur.

Choix techniques	
Backend & Base de données	Frontend
- Laravel 11 / PHP 8.3 - MySQL 8.0 (Eloquent ORM) - API REST sécurisée (clé per-agent)	- Blade (templates Laravel) - Chart.js (intégré local) - AJAX toutes les 30s
Agent de surveillance	Infrastructure de test
- Python 3.10 - Watchdog (inotify Linux) - SQLite (file locale offline) - psutil (processus)	3 VMs KVM/libvirt - Réseau 10.20.0.0/24 2FA TOTP RFC 6238 - Token usage unique (enrôlement)

Cycle de vie de l'agent

- 1 Démarrage : lecture .env
- 2 Enrôlement : token → clé API
- 3 Watchdog observe les dossiers
- 4 Événement → file SQLite locale
- 5 POST /api/agent/events
- 6 Heartbeat toutes les 30 s

Installation en une commande (Linux, Windows, macOS) :

```
curl -sSL http://<soc>/install.sh | bash
```

Scoring comportemental

Signal détecté	Score
Extension suspecte (.locked)	+80
Note de rançon créée	+55
Suppression clichés VSS	+60
LOLBin suspect	+45
Suppression massive fichiers	+50

Score < 30	Normal	
30 à 59	Suspect	
60 à 99	Élevé	Alerte + incident créés automatiquement en < 5 s
≥ 100	Critique	

2026-07-03

RansomShield
Réalisation et Résultats

Agent Python et moteur de détection

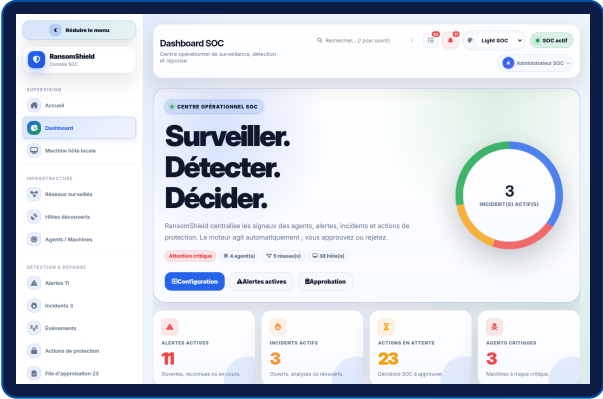
[AGENT & MOTEUR — 12 :15]

Au démarrage, l'agent s'enrôle via un token à usage unique qui lui donne une clé API permanente, puis il surveille les dossiers avec Watchdog et envoie un signal de présence toutes les trente secondes. Le moteur de détection calcule un score cumulatif configurable. Par exemple : un fichier renommé en .locked vaut 80 points, une note de rançon 55 points ; le total de 135 déclenche immédiatement une alerte critique et la création d'un incident. Tout ce processus prend moins de cinq secondes.

Agent Python et moteur de détection													
Cycle de vie de l'agent	Scoring comportemental												
1 Démarrage : lecture .env 2 Enrôlement : token → clé API 3 Watchdog observe les dossiers 4 Événement → file SQLite locale 5 POST /api/agent/events 6 Heartbeat toutes les 30 s	<table><tr><th>Signal détecté</th><th>Score</th></tr><tr><td>Extension suspecte (.locked)</td><td>+80</td></tr><tr><td>Note de rançon créée</td><td>+55</td></tr><tr><td>Suppression clichés VSS</td><td>+60</td></tr><tr><td>LOLBin suspect</td><td>+45</td></tr><tr><td>Suppression massive fichiers</td><td>+50</td></tr></table>	Signal détecté	Score	Extension suspecte (.locked)	+80	Note de rançon créée	+55	Suppression clichés VSS	+60	LOLBin suspect	+45	Suppression massive fichiers	+50
Signal détecté	Score												
Extension suspecte (.locked)	+80												
Note de rançon créée	+55												
Suppression clichés VSS	+60												
LOLBin suspect	+45												
Suppression massive fichiers	+50												
Installation en une commande (Linux, Windows, macOS) :	<table><tr><td>Score < 30</td><td>Normal</td></tr><tr><td>30 à 59</td><td>Suspect</td></tr><tr><td>60 à 99</td><td>Élevé</td></tr><tr><td>≥ 100</td><td>Critique</td></tr></table>	Score < 30	Normal	30 à 59	Suspect	60 à 99	Élevé	≥ 100	Critique				
Score < 30	Normal												
30 à 59	Suspect												
60 à 99	Élevé												
≥ 100	Critique												

Pages de la console

- 🏠 Tableau de bord
- 👤 Agents (état, heartbeat)
- 🔔 Alertes (score, signaux)
- 📁 Incidents
- 🕒 Timeline d'audit
- ✅ File d'approbation
- 🔧 Règles (CRUD live)
- 🔍 Recherche globale
- 🔔 Notifications
- ⚙️ Paramètres système



Mise à jour AJAX toutes les 30 s · Web · E-mail SMTP · Son · Webhook

2026-07-03

RansomShield
— Réalisation et Résultats

— Console SOC — Interface de supervision

[CONSOLE SOC — 13 :30]

La console SOC centralise dix pages fonctionnelles, avec un tableau de bord mis à jour automatiquement toutes les trente secondes. Le point important, c'est que toute action de protection passe par une file d'approbation : l'analyste doit explicitement valider ou rejeter chaque action avant qu'elle ne s'exécute. Les notifications partent simultanément par quatre canaux : le web, l'e-mail, le son, et un webhook. Vous verrez tout cela dans la démonstration.



Tests de validation et résultats

5 scénarios d'attaque testés

Scénario	Évts	Résultat
Chiffrement basique	7	✓
Kill chain complète	22	✓
Chiffrement massif	18	✓
Exfiltration données	8	✓
LOLBins suspects	5	✓

VMs KVM — réseau 10.20.0.0/24

Environnement : 3

Résultats obtenus

- ✓ Détection en < 5 secondes (tous scénarios)
- ✓ Aucun événement perdu (file SQLite)
- ✓ Pipeline complet validé : événement → alerte → incident → timeline
- ✓ Rollback isolation réseau fonctionnel
- ✓ 2FA TOTP et API sécurisée (per-agent)

Limite identifiée

Tests conduits sur Linux uniquement. Un attaquant ralentissant son rythme réduit la visibilité.

2026-07-03

RansomShield

Réalisation et Résultats

Tests de validation et résultats

[TESTS & RÉSULTATS — 14 :45]

Nous avons validé le système sur cinq scénarios d'attaque, de sept à vingt-deux événements. Tous ont été détectés en moins de cinq secondes, sans perte d'événement, avec un pipeline complet validé de bout en bout, de l'événement jusqu'à la timeline. Je précise honnêtement une limite : les tests ont été conduits uniquement sur Linux, et un attaquant qui ralentirait son rythme resterait moins visible. Passons maintenant à la démonstration du système.

5 scénarios d'attaque testés			Résultats obtenus
Scénario	Évts	Résultat	
Chiffrement basique	7	✓	Résultats obtenus • Détection en < 5 secondes (tous scénarios) • Aucun événement perdu (file SQLite) • Pipeline complet validé : événement → alerte → incident → timeline • Rollback isolation réseau fonctionnel • 2FA TOTP et API sécurisée (per-agent)
Kill chain complète	22	✓	
Chiffrement massif	18	✓	
Exfiltration données	8	✓	
LOLBins suspects	5	✓	
VMs KVM — réseau 10.20.0.0/24			
			Limite identifiée Tests conduits sur Linux uniquement. Un attaquant ralentissant son rythme réduit la visibilité.



Démonstration Live

5 minutes — système réel sur VMs KVM

2026-07-03

RansomShield
— Réalisation et Résultats

▶ Démonstration Live

5 minutes — système réel sur VMs KVM

[DÉMO LIVE — 15 :00, checklist 5 min, hors chrono présentation]

1. Ouvrir `http://10.20.0.1` (console SOC)
2. Se connecter : identifiants + **code 2FA**
3. Vérifier : 3 agents en ligne (heartbeat vert)
4. **Agents** → VM-02 → **Lancer simulation** → "Kill chain complète" (22 évts)
5. **Dashboard** : le score monte en temps réel
6. **Alertes** : niveau Critique + signaux détectés
7. **Incidents** : ouvrir l'incident auto-crée
8. **File d'approbation** : approuver l'isolation réseau
9. **Timeline** : tout est horodaté et tracé

Rester calme, ne pas se précipiter — chaque étape a le temps qu'il faut.



Conclusion et Perspectives



Conclusion

RansomShield est une solution opérationnelle couvrant l'ensemble du cycle de traitement d'un incident : **collecte** → **analyse** → **alerte** → **réponse** → **audit**.

- ✓ Agent Python multi-plateforme — installation en une commande
- ✓ Console SOC : simulation, 4 canaux de notification, rapports PDF, 2FA
- ✓ Moteur de détection : scoring configurable, règles CRUD, < 5 s
- ✓ Environnement contrôlé : aucun vrai malware déployé

Perspectives d'évolution

- **HTTPS** — chiffrement TLS en production
- **RBAC** — rôles distincts (administrateur, analyste SOC)
- **Machine Learning** — Isolation Forest (baseline comportementale)
- **MITRE ATT&CK** — alignement et couverture du référentiel
- **Intégration SIEM** — export Syslog vers Splunk / ELK Stack

2026-07-03

RansomShield
└─ Conclusion et Perspectives

└─ Conclusion et Perspectives

[CONCLUSION & PERSPECTIVES — 20 :00]

En résumé, RansomShield est une solution fonctionnelle qui couvre tout le cycle de traitement d'un incident : collecte, analyse, alerte, réponse et audit. Elle repose sur un agent multi-plateforme, une console complète, un moteur de détection réactif, et un contrôle humain à chaque étape. Ce travail a ses limites : il a été testé dans un environnement contrôlé, avec des réponses en partie simulées. Les perspectives sont claires : ajouter le chiffrement HTTPS, des rôles distincts pour les utilisateurs, puis à terme du machine learning pour affiner la détection, et une intégration avec les outils SIEM du marché. Je vous remercie pour votre attention et reste à votre disposition pour vos questions.

Conclusion et Perspectives	
Conclusion	
RansomShield est une solution opérationnelle couvrant l'ensemble du cycle de traitement d'un incident : collecte → analyse → alerte → réponse → audit.	
✓	Agent Python multi-plateforme — installation en une commande
✓	Console SOC : simulation, 4 canaux de notification, rapports PDF, 2FA
✓	Moteur de détection : scoring configurable, règles CRUD, < 5 s
✓	Environnement contrôlé : aucun vrai malware déployé
Perspectives d'évolution	
➤	HTTPS — chiffrement TLS en production
➤	RBAC — rôles distincts (administrateur, analyste SOC)
➤	Machine Learning — Isolation Forest (baseline comportementale)
➤	MITRE ATT&CK — alignement et couverture du référentiel
➤	Intégration SIEM — export Syslog vers Splunk / ELK Stack

Merci de votre aimable attention !



Codjo Fréjus Loïc SANGRONIO

lsangronio@gmail.com | IFRI-UAC 2023–2024

Encadreur : Ing. Guy KPEDJO

2026-07-03

RansomShield
— Conclusion et Perspectives

[QUESTIONS DU JURY — aide-mémoire]

- **Pourquoi Python et pas C/Go ?** → Watchdog exploite inotify nativement, CPU minimal, portable Linux/Windows/macOS
- **Faux positifs ?** → seuils configurables, **opérateur humain** valide ou rejette chaque action
- **Pourquoi pas HTTPS ?** → LAN contrôlé ; c'est la 1^{re} perspective pour la prod
- **Différence avec un antivirus ?** → comportement, pas signature ; fonctionne sur variantes inconnues
- **Testé sur Windows ?** → Non, limite assumée : Linux uniquement

Garder les ***annexes*** prêtes (file d'approbation, captures d'écran).

Merci de votre aimable attention !



Codjo Fréjus Loïc SANGRONIO
lsangronio@gmail.com | IFRI-UAC 2023–2024
Encadreur : Ing. Guy KPEDJO

Principe

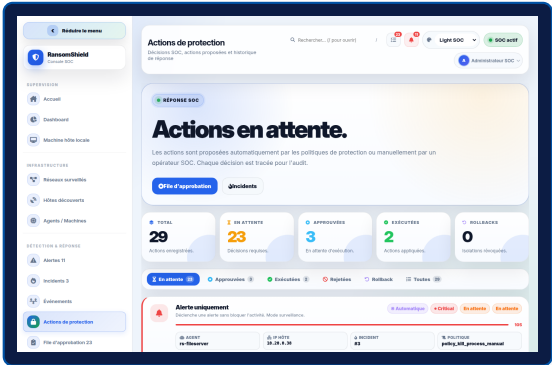
Le moteur **propose** — l’opérateur **décide**.

Chaque action affiche :

- Type (scan, isolation, quarantaine)
- Niveau de risque de l’incident
- Agent concerné

Boutons **Approuver** / **Rejeter** — tout est horodaté.

Rollback possible sur l’isolation réseau.



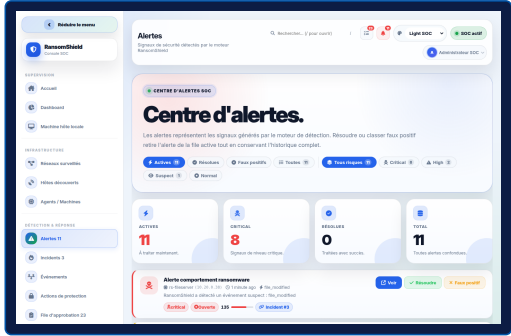
2026-07-03

Annexe A — File d’approbation et rollback



Annexe B — Captures d'écran

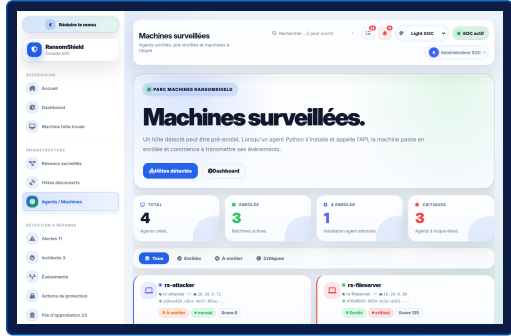
Alertes



Timeline incident



Agents



Simulation en cours



RansomShield

2026-07-03

Annexe B — Captures d'écran

